

重新思考蒸馏数据集的存储方式：SPEED

NeurIPS 2023 · 一种面向 epitomic 数据集蒸馏的稀疏共享参数化方法，用极小的存储代价刷新 SOTA

现代深度学习依赖庞大的数据集，而这些数据集的存储、传输和训练成本都很高。数据集蒸馏（dataset distillation）提供了一条诱人的捷径：把一个大数据集压缩成一个极小的合成数据集，却仍能训练出精度可用的模型。难点在于合成预算极其有限，有时每个类别只允许 1 张图像，因此花在合成集上的每一个参数都必须物尽其用。

这一领域的多数工作都把精力投入到匹配目标（matching objective）上，也就是通过梯度、特征分布或训练轨迹让合成数据对齐真实数据的那个损失函数。至于合成图像本身如何被参数化，则一直被当作次要问题：通常每张图像都被独立优化，作为一块互不相关的像素或特征。这种朴素方案悄悄忽略了一个计算机视觉研究者都熟知的事实：自然图像充满冗余。纹理会重复，图像块彼此呼应，相似的结构在许多图像、许多类别之间反复出现。

SPEED（发表于 NeurIPS 2023）认真对待了这种冗余。它的全称是 Sparse Parameterization for Epitomic Dataset Distillation，核心思想是把合成数据集重新表述为「一个共享字典加上一组稀疏的逐图像编码」，借用了字典学习与稀疏编码的经典思路。具体来说，它引入三个部件：Spatial-Agnostic Epitomic Tokens（SAETs，空间无关的表征性 token），一小组被所有图像块反复复用、与位置无关的 token；Sparse Coding Matrices（SCMs，稀疏编码矩阵），为每张图像只挑选最重要的少数 token；以及 Feature-Recurrent Network（FReeNet，特征循环网络），把这些 token 重新组装成完整的高分辨率合成图像。

框架总览

下图追踪了一张合成图像在整条流水线中的旅程。左侧，一组共享的 SAETs（画成 H 个并行的 head）与逐图像的 SCM 相结合，其中只有少数元素非零。这些稀疏组合被送入 FReeNet：一个紧凑的、类 Transformer 的模块，包含 Add & Norm 与 MLP 层，并被循环地重复应用，在每一步都复用同一套共享 token 和编码，从而构建出层次化的图像块。右侧合成出的图像，正是匹配目标真正拿来与真实数据比较的对象。

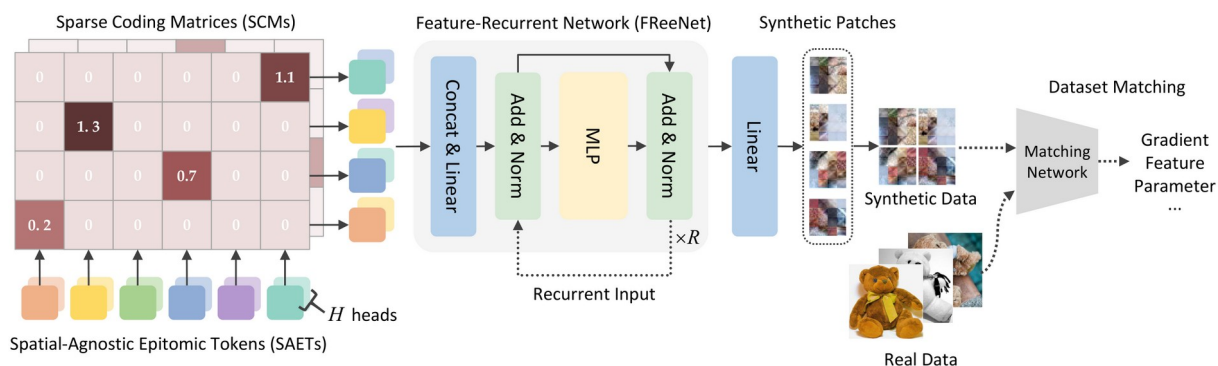


图 1 SPEED 框架：共享的 Spatial-Agnostic Epitomic Tokens 字典与稀疏的逐图像编码矩阵，经由 FReeNet 循环组装成合成图像，再与真实数据进行匹配。

论 文：https://proceedings.neurips.cc/paper_files/paper/2023/hash/9e8889198d16fb79926e71adbe38cae4-Abstract-Conference.html
 码：<https://github.com/MIV-XJTU/SPEED> 代

问题：冗余藏在参数化里

要理解参数化为何重要，不妨设想最极限的设置：在 CIFAR100 上每个类别只有 1 张图像。朴素方法会把全部预算用来逐像素地画出每类恰好 1 张合成图像，类别之间毫无共享。然而这些像素中有大量都在反复编码相同的边缘、纹理和色块。近来已有少数方法开始利用合成图像之间的关系，但没有一个以「空间无关」的方式去除冗余，也就是不管某个特征出现在哪里。SPEED 的赌注是：如果几乎不把预算花在共享字典和逐图像稀疏编码上，再让一个小网络去重建其余部分，就能在相同存储里塞进多得多的有用信息。

SPEED 如何参数化一个数据集

每张合成图像都被写成两样东西的函数：对整个数据集都相同的共享 SAETs 字典，以及一个逐图像的 Sparse Coding Matrix，用来挑选并加权少数 token。多头稀疏编码把二者结合，于是一张图像被表示为共享基元的稀疏混合，而不再是一格孤立的像素。由于字典和网络在所有图像间共享，每张图像真正需要存储的，只是它那份很短的非零编码列表。

随后 FReeNet 把这些稀疏特征变成图像。它是一个紧凑的、类 Transformer 的网络（带残差连接和层归一化的 MLP），在 R 个块上循环应用，让一小组权重就能合成出层次化的高分辨率图像块。训练时最小化所选的匹配目标，并加上一个 L1 稀疏惩罚，把大多数编码元素推向零。训练结束后，每个编码矩阵都被剪枝到其 top-k 个非零元素，并以稀疏格式存储，于是编码开销只随这少数几个元素增长，而不随整个字典增长。

收益是实打实的。在同一个 CIFAR100 预算下，朴素方法只够画出每类 1 张图像，而 SPEED 凭借共享字典和稀疏编码，能为每个类别留出约 11 张合成图像的空间。更多图像，又都来自一个可复用的共享表征，就意味着下游模型能学到更多有用信息。

结果：用零头的存储刷新 SOTA

SPEED 在三个标准基准上评测：32×32 的 CIFAR10 与 CIFAR100、64×64 的 TinyImageNet，外加六个 128×128 的高分辨率 ImageNet 子集（ImageNette、ImageWoof、ImageFruit、ImageMeow、ImageSquawk、ImageYellow），每个子集包含 10 个类别。所有比较都在相同存储预算下进行，以每类参数量计，覆盖 IPC 1、10、50。在所有这些设置上，SPEED 都刷新了 state-of-the-art (SOTA)。

表 1 SPEED 在极限存储预算下的核心结果。

基准	设置	SPEED 准确率	相比先前 SOTA
CIFAR100	IPC 1	40.0%	+6.0%
TinyImageNet	IPC 1	26.9%	+10.9%
ImageNet 子集 (平均)	IPC 1	—	+11.2%
ImageSquawk	IPC 10	71.8%	+15.0%

这些数字值得读两遍。在最紧的 IPC 1 预算下，SPEED 在 CIFAR100 上达到 40.0%，比先前最好成绩高出 +6.0%；在 TinyImageNet 上达到 26.9%，高出 +10.9%。在 ImageNet 子集上，它在相同预算下相比先前 SOTA 平均提升 +11.2%。最惊人的是，它的 IPC 1 结果足以媲美以往方法在 IPC 10 时的表现，而只用了大约 10% 的参数。在 ImageSquawk 的 IPC 10 上，它达到 71.8%，跃升 +15.0%。这些合成数据还能迁移到未见过的骨干网络：在 CIFAR10 IPC 10 上用 ViT 达到 51.5%，提升 +3.6%。

这些收益在压力测试下依然成立。在类别分步到来的持续学习（continual learning）设置中，SPEED 在每一个阶段都保持领先。下图追踪了测试准确率随类别数量增长的变化，分别在 ConvNet 和 ResNet18 上；SPEED 的曲线自始至终都高于 DM、HaBa 和 IDC。

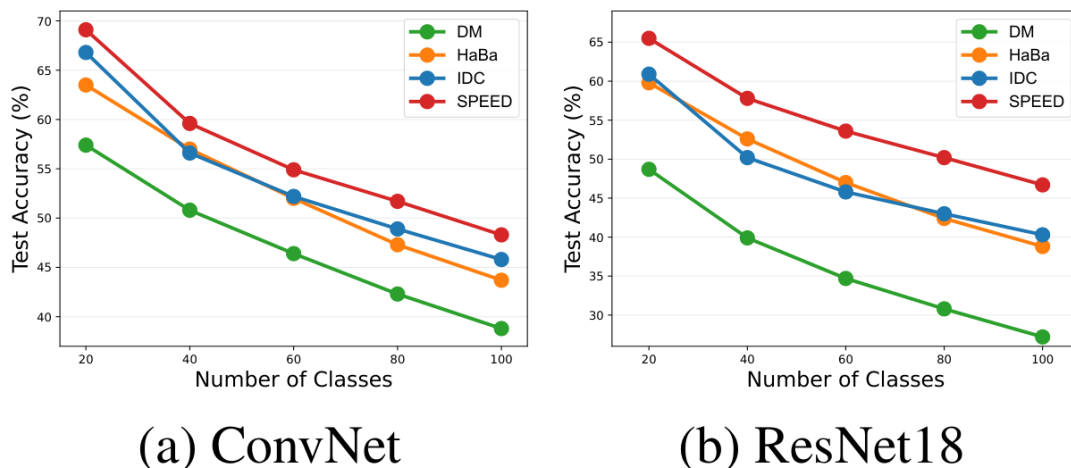


图 2 持续学习：无论在 ConvNet 还是 ResNet18 上，SPEED 在每一步都保持最佳测试准确率。

鲁棒性讲了同样的故事。作者用 SPEED 的合成数据训练模型，再在损坏（corruption）下测试，横跨 14 种损坏类型、5 个严重程度比较准确率。在 ResNet18 上，SPEED 在几乎每一种损坏下都把先前方法的准确率提高了近一倍。

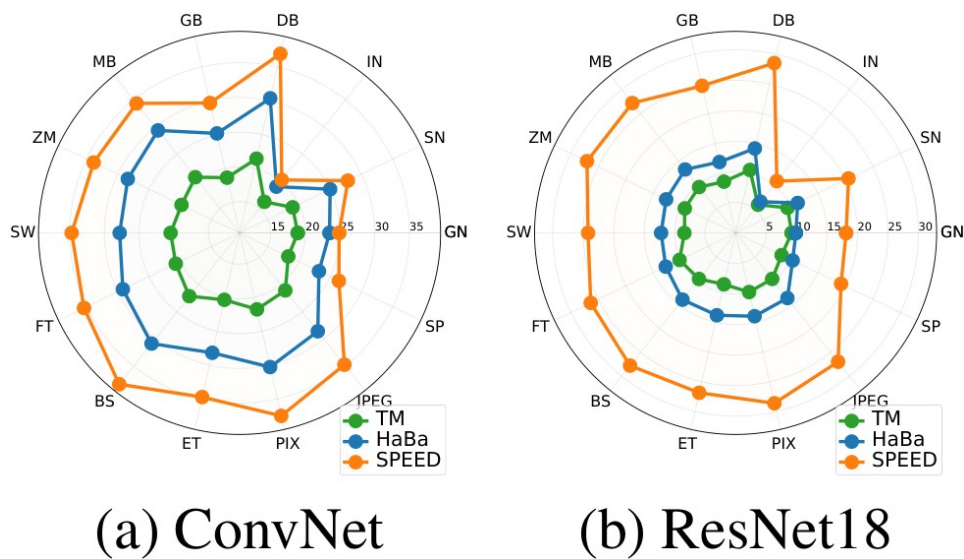


图 3 面对多种损坏类型的鲁棒性：无论在 ConvNet 还是 ResNet18 上，SPEED（橙色）都明显优于 TM 与 HaBa。

稀疏化几乎是免费的

一个自然的担忧是：如此激进地剪枝编码，会不会毁掉合成数据？SPEED 的消融实验给出了否定的答案。把每个编码矩阵剪枝到只剩 $k=48$ 个非零元素（约 307K 参数），既满足存储预算，又让 ConvNet 的准确率保持在 73.5%，仅略低于完整的 15M 参数模型的 74.0%。若把剪枝推得太狠，降到 $k=12$ 、约 106K 参数，准确率确实会崩到 57.8%，所以适中的 k 才是甜点，而且它还能改善对未见架构的泛化。在网络一侧，2 个循环块（ $R=2$ ）和 3 个注意力头（ $H=3$ ）表现最好。

最有说服力的是定性检查。下图展示了把稀疏度压到 0.3% 密度前后的 ImageSquawk 合成样本。两者几乎无法区分：剪枝去掉了几乎所有存储的数值，却保留了每张图像的全局语义。

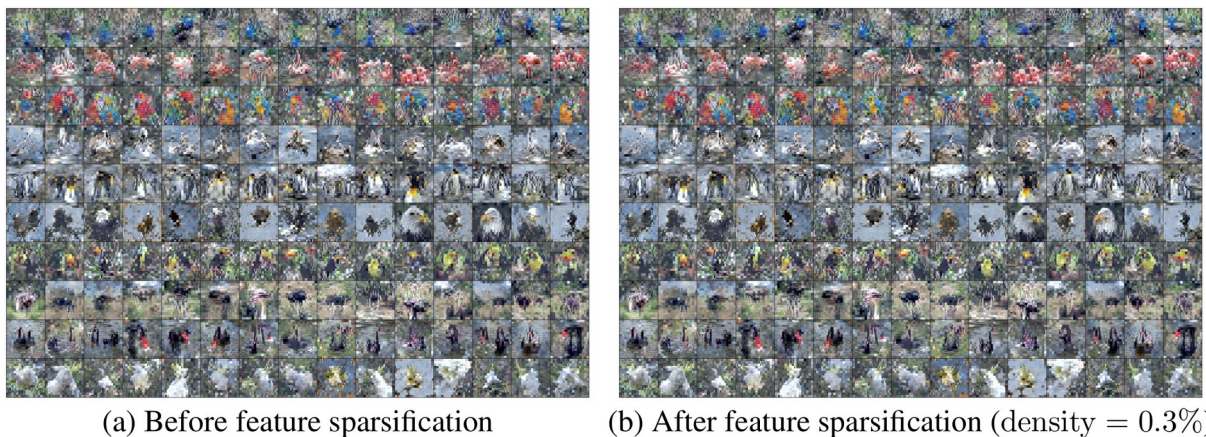


图 4 ImageSquawk 在特征稀疏化到 0.3% 密度前后的合成样本：剪枝后的图像与原图几乎一模一样。

为什么这很重要

SPEED 留下的长久启示是：如何参数化一个合成数据集，值得与你所优化的匹配目标同等的重视。通过把数据集看作「一个空间无关的共享 token 字典、稀疏的逐图像编码，以及一个负责重新组装的小型循环网络」，SPEED 去除了朴素方法白白浪费的空间冗余。结果就是用零头的存储实现了 state-of-the-art 的蒸馏，在高分辨率数据上收益最大，同时对未见架构有更好的迁移、对损坏有更强的鲁棒性。它还是一个即插即用的想法：因为它是一种参数化而非新的目标，所以能同样地嵌入梯度匹配、分布匹配和轨迹匹配。换句话说，稀疏而共享的表征，是让微小合成数据集发挥更大价值的一个通用杠杆。

这一切并非完全免费。SPEED 在蒸馏流程中额外引入了一个小字典和一个循环合成器，而稀疏程度 k 也需要谨慎选择，因为剪枝过猛确实会牺牲精度。但这一框架不要求改动匹配目标，也为更丰富的字典、可学习的稀疏模式和更大规模的数据集留下了空间。对任何要构建紧凑合成数据的人来说，实践启示很简单：把参数化当作第一等的设计决策，而不是在确定目标之后才顺手填上的细节。